

IOT

Parte II

Adriana Palacios 23044 · Bryan Martínez 23542







PASO 0 · QUICKSTART

1. python -m venv .venv && source .venv/bin/activate
2. pip install -r requirements.txt -> kafka-python, matplotlib, numpy
3. python -m producer.producer --dry-run -> genera una lectura sin conectarse a Kafka
4. python -m consumer.consumer -> terminal 1: escucha el topic y grafica
5. python -m producer.producer -> terminal 2: publica una lectura cada 15 a 30 s

Cabe mencionar que los parámetros de conexión se centralizaron en config/kafka_settings.py, de tal forma que producer y consumer comparten la misma configuración. Abajo se muestra la salida del dry-run, con la que validamos que el entorno estuviera listo antes de conectarnos al broker.

```
Topic: 23542 | Key: sensor1
Medición: {'temperature': 48.3, 'humidity': 56, 'wind_direction': 'N'}
Payload: {"temperature": 48.3, "humidity": 56, "wind_direction": "N"}
Tamaño: 60 bytes
```

3.1 SIMULACIÓN DEL SENSOR

. Temperatura entre 0 y 110 grados centígrados, float de 2 decimales, normal ($\mu = 55$, $\sigma = 15$).
. Humedad entre 0 y 100 %, entero, normal ($\mu = 55$, $\sigma = 15$).
. Viento en una de 8 direcciones {N, NO, O, SO, S, SE, E, NE}, uniforme.

Si un valor cae fuera de su rango, se vuelve a generar, para poder garantizar que todas las lecturas respeten los límites de cada sensor.

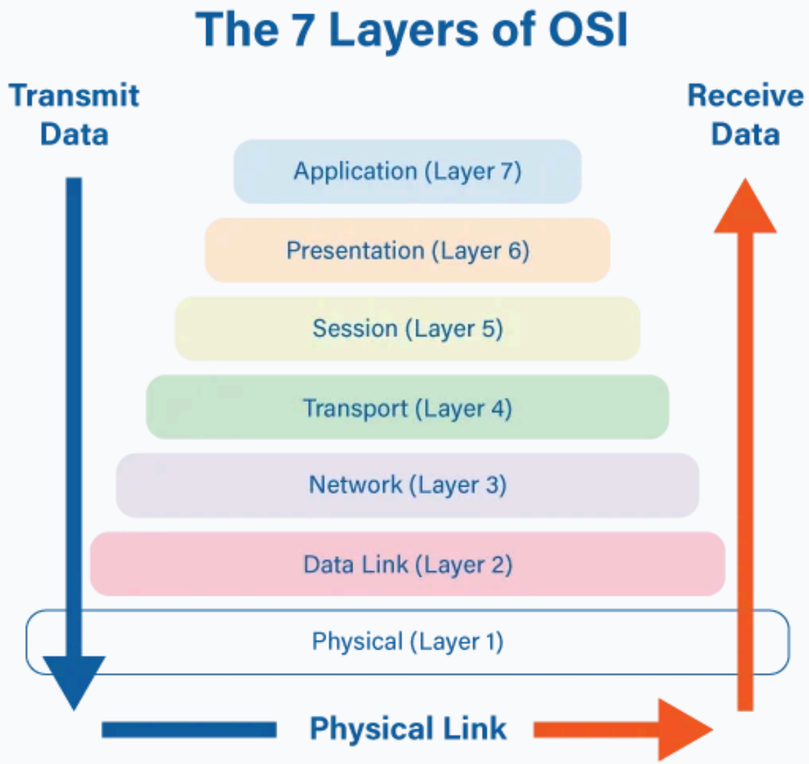
Luego, cada lectura se empaqueta en un JSON de 60 bytes, por ejemplo {"temperature": 48.3, "humidity": 56, "wind_direction": "N"}



¿A QUÉ CAPA PERTENECE JSON/SOAP SEGÚN EL MODELO OSI Y POR QUÉ?

Consideramos que pertenecen a la capa de presentación, ya que su función es definir cómo se representan y estructuran los datos, de tal forma que emisor y receptor los interpreten de la misma manera.

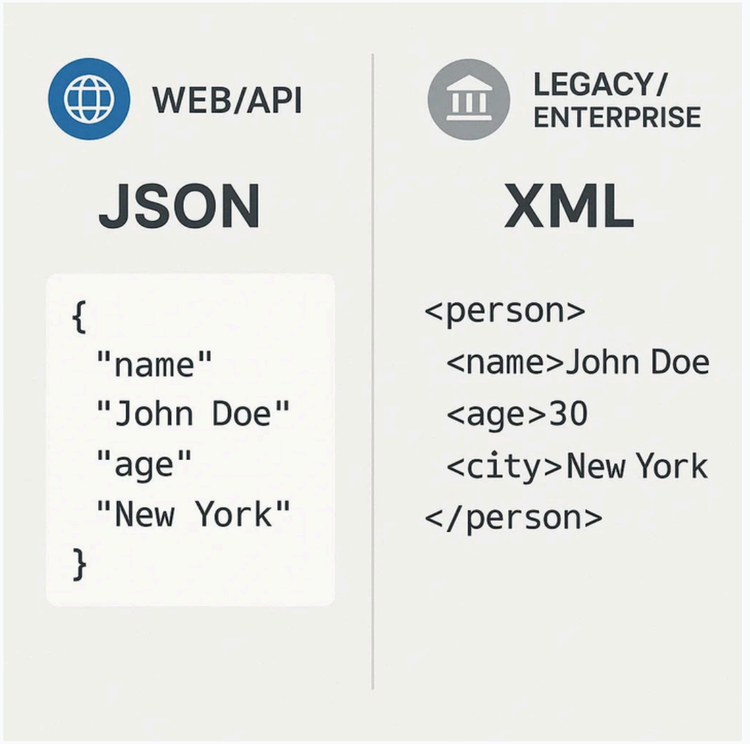
Por otro lado, no pertenecerían a las capas inferiores, pues no se encargan de la transmisión física ni del enlace, tampoco de direccionar o enrutar paquetes (red) ni de garantizar la entrega entre extremos (transporte). Asimismo, diría que tampoco son de la capa de aplicación, porque no definen el servicio de comunicación como tal.



¿QUÉ BENEFICIOS TIENE UTILIZAR UN FORMATO COMO JSON/SOAP?

Ambos son prácticamente estándares para el intercambio de datos. Por un lado, SOAP es un protocolo basado en XML con una estructura de mensaje definida (envelope, header y body), lo que permite validar los mensajes contra un esquema, y es por esto que suele utilizarse en servicios que requieren contratos estrictos.

Por otro lado, JSON es uno de los formatos más comunes a la hora de interactuar con APIs, datasets o recursos de distintas fuentes, es ligero, legible y casi cualquier lenguaje lo soporta de forma nativa. En nuestro caso, esto facilitó bastante el manejo de las lecturas, pues el producer las serializa y el consumer las interpreta sin necesidad de saber cómo fueron generadas.



3.2 ENVÍO DE DATOS - SINTAXIS EN PYTHON

```
producer = KafkaProducer(
   bootstrap_servers="161.35.11.141:9092",
   security_protocol="SASL_PLAINTEXT",
   sasl_mechanism="PLAIN",
   sasl_plain_username="username",
   sasl_plain_password="********",
)
producer.send("23542", key=b"sensor1", value=payload).get(timeout=15)
```

Utilizamos la librería kafka-python, por lo que la conexión se configura directamente al instanciar el KafkaProducer con los parámetros de autenticación de la guía. Cabe mencionar que el envío se hace con .get(), de tal forma que el producer espera la confirmación del broker y puede mostrar en qué partición y offset quedó cada lectura.

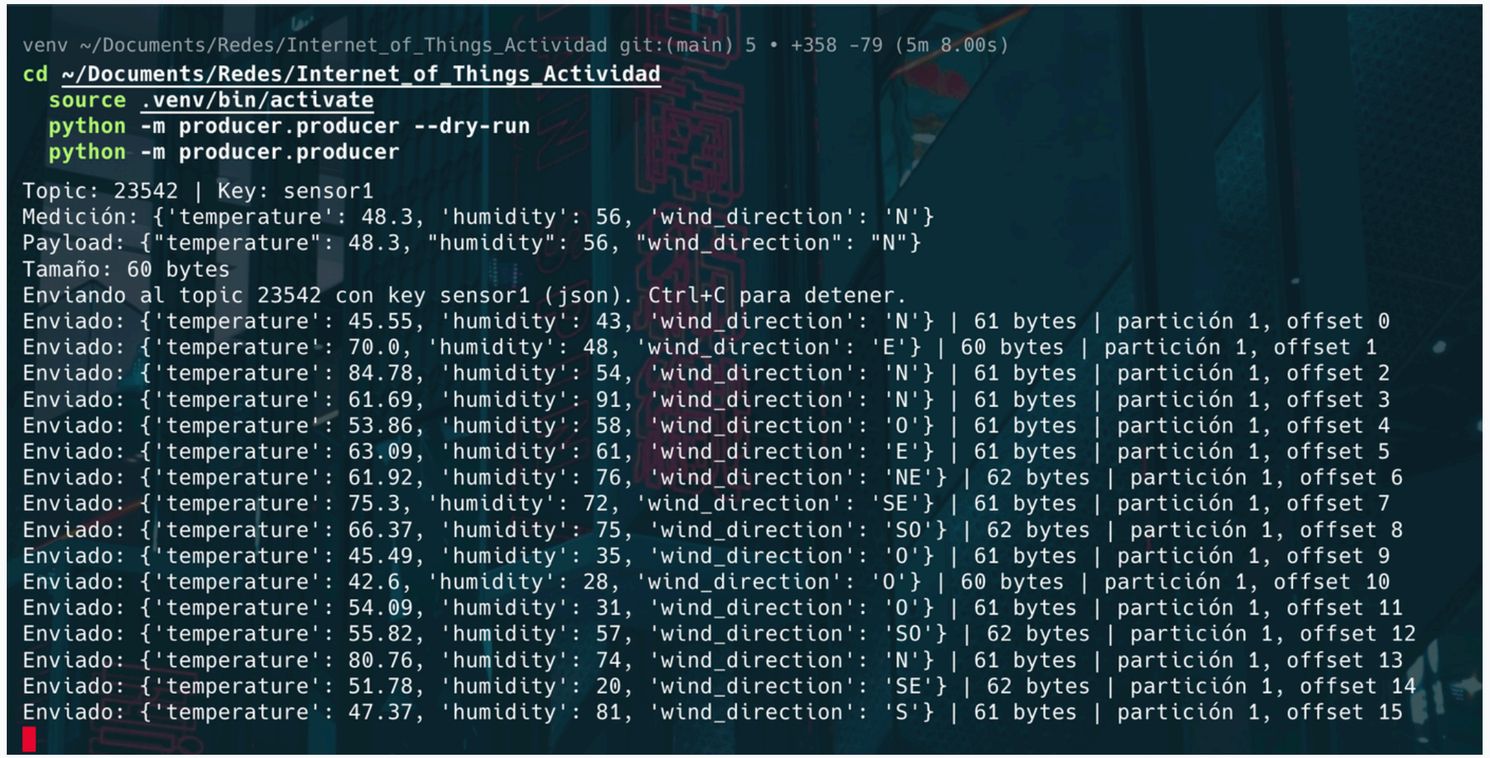
3.2 ENVÍO DE DATOS - PSEUDOCÓDIGO

```
ALGORITMO ProductorEstacionMeteorologica
ENTRADA: servidor <- "161.35.11.141:9092", topic <- "23542", key <- "sensor1"

01  productor <- CONECTAR_KAFKA(servidor, SASL_PLAINTEXT, PLAIN, credenciales)
02  MIENTRAS el programa no sea interrumpido HACER
03      temperatura <- MUESTREAR_NORMAL(μ = 55, σ = 15) en [0, 110], 2 decimales
04      humedad <- MUESTREAR_NORMAL(μ = 55, σ = 15) en [0, 100], entero
05      viento <- ELEGIR_UNIFORME({N, NO, O, SO, S, SE, E, NE})
06      mensaje <- CODIFICAR_JSON(temperatura, humedad, viento)
07      confirmación <- ENVIAR(productor, topic, key, mensaje)
08      MOSTRAR(mensaje, confirmación.partición, confirmación.offset)
09      ESPERAR(aleatorio entre 15 y 30 segundos)
10  FIN MIENTRAS
11  CERRAR(productor)
```

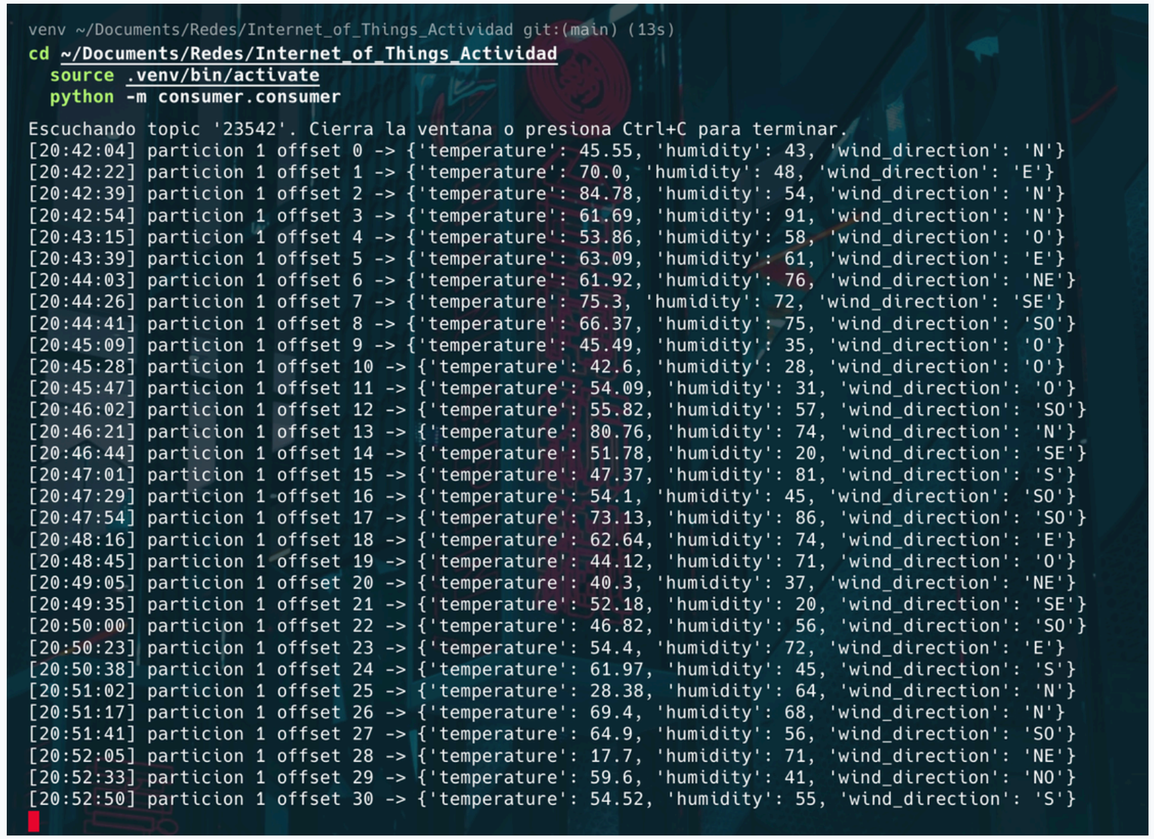


3.2 ENVÍO DE DATOS - EVIDENCIA



En la captura se observa al producer publicando en el topic 23542 con la key sensor1. Cada lectura pesa entre 60 y 62 bytes y el broker confirma que quedó en la partición 1, con offsets del 0 al 15, enviándose cada 15 a 30 segundos.

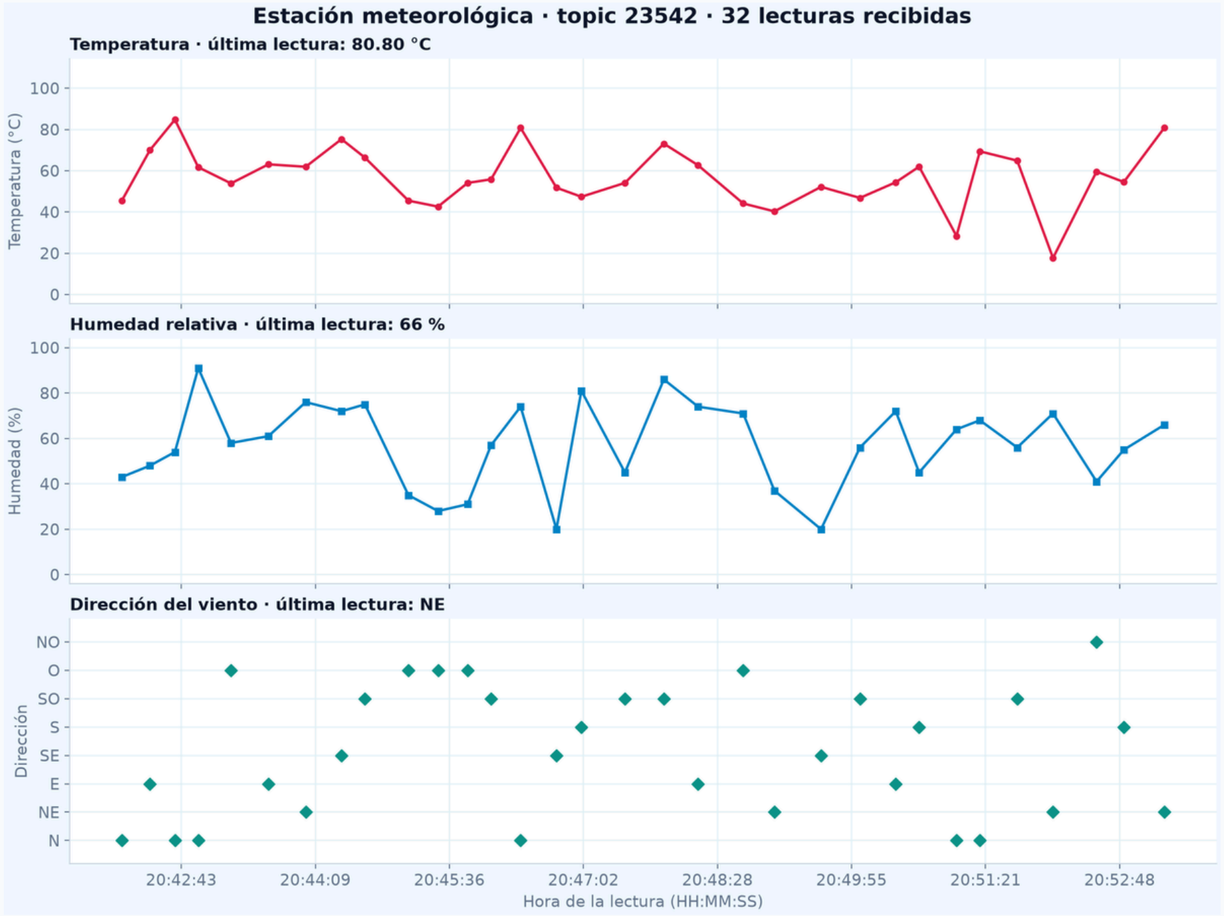
3.3 CONSUMO DE DATOS - CONSUMER



El consumer se suscribe al topic 23542 con las mismas credenciales del producer y, cada vez que llega un mensaje, lo decodifica, actualiza la gráfica y guarda una imagen de ella. Al leer desde el inicio del topic, la gráfica muestra tanto la telemetría pasada como la entrante.

En la captura se observa que las lecturas llegan en el mismo orden en que fueron enviadas (partición 1, offsets del 0 al 30) y con los mismos valores que publicó el producer.

3.3 CONSUMO DE DATOS - GRÁFICA EN TIEMPO REAL



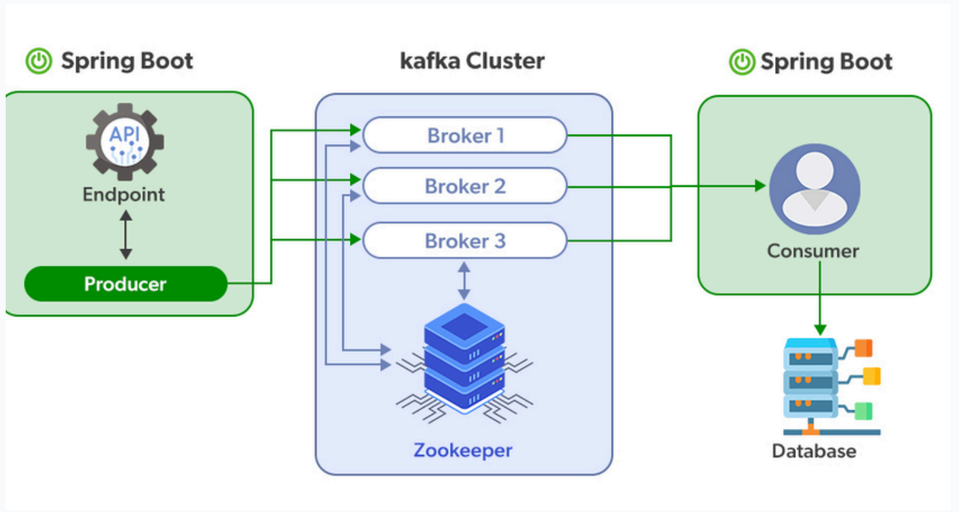
La gráfica se actualiza cada vez que llega una nueva lectura (aproximadamente cada 15 a 30 s) y muestra en tres paneles la temperatura, la humedad relativa y la dirección del viento a lo largo del tiempo.

En este caso se registraron 32 lecturas, y se puede observar cómo la temperatura y la humedad oscilan alrededor de 55, acorde a la distribución normal utilizada en el sensor.

¿QUÉ VENTAJAS Y DESVENTAJAS CONSIDERA QUE TIENE ESTE ACERCAMIENTO BASADO EN PUB/SUB DE KAFKA?

La principal ventaja está en que no nos limitamos a una única fuente registrando o publicando los datos, sino que se tiene prácticamente un flujo constante de información. Al ser un publicador al que se puede conectar cualquier consumidor con las credenciales correctas, cada uno de estos puede generar aplicaciones, tomar decisiones o presentar los datos de la forma en que lo requiera.

No obstante, esto genera cierta complejidad, ya que hay que mantener el broker, configurar la autenticación y manejar conceptos como particiones y offsets. Además, todo depende de que el broker esté disponible; cuando el servidor dejó de responder, no pudimos enviar ni consumir datos.



¿PARA QUÉ APLICACIONES TIENE SENTIDO USAR KAFKA? ¿PARA CUÁLES NO?

Tiene sentido cuando se genera un flujo continuo de datos que debe procesarse en tiempo real y que distintos sistemas necesitan consumir, mientras que no aporta tanto cuando la interacción es puntual o el volumen de datos es bajo.

Sí aplica

- Telemetría IoT y sensores
- Monitoreo de logs y métricas
- Alertas por umbrales (p. ej. temperatura alta)
- Transacciones o pagos en tiempo real

No aplica

- Peticiones puntuales (iniciar sesión, consultar un dato)
- Aplicaciones pequeñas con poco tráfico
- Reemplazo de una base de datos para hacer consultas
- Envío de archivos grandes

3.4 RESTRICCIONES - CÓMO LO ABORDAMOS

El Edge Server del curso dejó de estar disponible para esta fase, por lo que replicamos su configuración en un broker Kafka local con Docker, conservando lo siguiente.

- Mismo protocolo SASL_PLAINTEXT con mecanismo PLAIN.
- Mismas credenciales.
- Mismo topic 23542 y key sensor1.

El producer y el consumer solo cambian la dirección del servidor, el resto del flujo es igual al de las fases anteriores. Para correrlo se levanta el broker con docker compose up -d dentro de fase_compacta_local, y luego se ejecutan python -m fase_compacta_local.consumer y python -m fase_compacta_local.producer en terminales separadas.

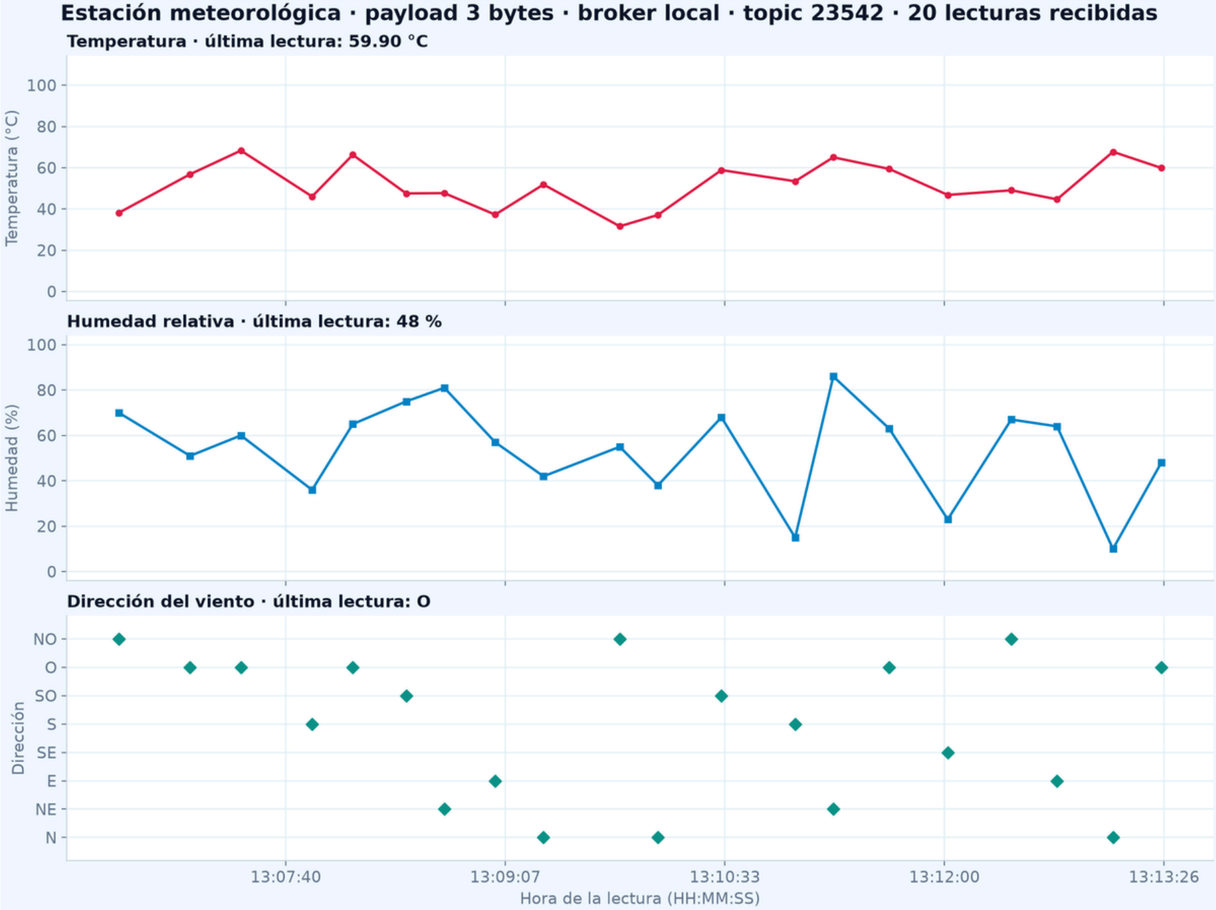
3.4 RESTRICCIONES - CODIFICACIÓN EN 3 BYTES

[temperatura × 100 : 14 bits] [humedad : 7 bits] [viento : 3 bits]

Encode toma la lectura en JSON y la empaqueta en 24 bits; decode hace el proceso inverso y devuelve el mismo JSON.

JSON original: {"temperature": 77.23, "humidity": 33, "wind_direction": "N"} (61 bytes)
Payload codificado: 0x78ad08 (3 bytes)
JSON decodificado: {"temperature": 77.23, "humidity": 33, "wind_direction": "N"}

3.4 RESTRICCIONES - EVIDENCIA



Se enviaron y recibieron 20 lecturas de 3 bytes cada una (offsets 0 a 19), y el consumer las decodificó y graficó con los mismos valores que publicó el producer.

3.4 RESTRICCIONES - ENVÍO Y CONSUMO

Extracto de los logs de la corrida (primeras 8 de 20 lecturas). Cada payload que envió el producer llega igual al consumer, que lo decodifica de vuelta al mismo JSON.

producer_log.txt

```
Enviando payloads de 3 bytes a localhost:9092, topic 23542, key sensor1. Ctrl+C para detener.
Enviado: {"temperature": 38.17, "humidity": 70, "wind_direction": "NO"} -> 0x3ba631 | 3 bytes | partición 1, offset 0
Enviado: {"temperature": 56.84, "humidity": 51, "wind_direction": "O"} -> 0x58d19a | 3 bytes | partición 1, offset 1
Enviado: {"temperature": 68.32, "humidity": 60, "wind_direction": "O"} -> 0x6ac1e2 | 3 bytes | partición 1, offset 2
Enviado: {"temperature": 46.04, "humidity": 36, "wind_direction": "S"} -> 0x47f124 | 3 bytes | partición 1, offset 3
Enviado: {"temperature": 66.26, "humidity": 65, "wind_direction": "O"} -> 0x678a0a | 3 bytes | partición 1, offset 4
Enviado: {"temperature": 47.53, "humidity": 75, "wind_direction": "SO"} -> 0x4a465b | 3 bytes | partición 1, offset 5
Enviado: {"temperature": 47.68, "humidity": 81, "wind_direction": "NE"} -> 0x4a828f | 3 bytes | partición 1, offset 6
Enviado: {"temperature": 37.26, "humidity": 57, "wind_direction": "E"} -> 0x3a39ce | 3 bytes | partición 1, offset 7
```

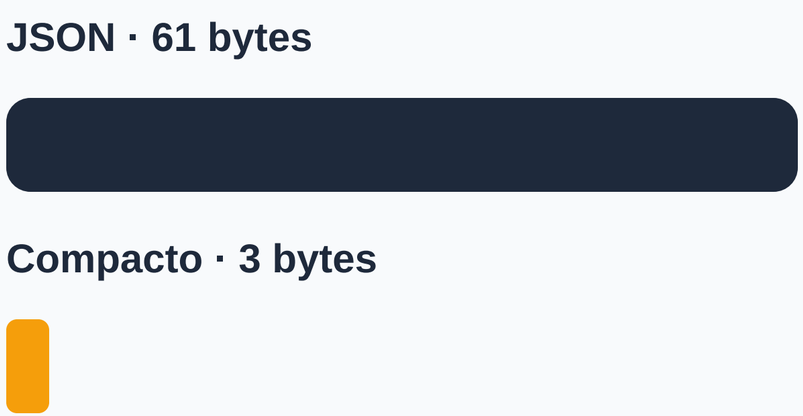
consumer_log.txt

```
Escuchando topic '23542' en localhost:9092. Cierra la ventana o presiona Ctrl+C para terminar.
[13:06:35] particion 1 offset 0 | 0x3ba631 (3 bytes) -> {'temperature': 38.17, 'humidity': 70, 'wind_direction': 'NO'}
[13:07:03] particion 1 offset 1 | 0x58d19a (3 bytes) -> {'temperature': 56.84, 'humidity': 51, 'wind_direction': 'O'}
[13:07:23] particion 1 offset 2 | 0x6ac1e2 (3 bytes) -> {'temperature': 68.32, 'humidity': 60, 'wind_direction': 'O'}
[13:07:51] particion 1 offset 3 | 0x47f124 (3 bytes) -> {'temperature': 46.04, 'humidity': 36, 'wind_direction': 'S'}
[13:08:07] particion 1 offset 4 | 0x678a0a (3 bytes) -> {'temperature': 66.26, 'humidity': 65, 'wind_direction': 'O'}
[13:08:28] particion 1 offset 5 | 0x4a465b (3 bytes) -> {'temperature': 47.53, 'humidity': 75, 'wind_direction': 'SO'}
[13:08:43] particion 1 offset 6 | 0x4a828f (3 bytes) -> {'temperature': 47.68, 'humidity': 81, 'wind_direction': 'NE'}
[13:09:03] particion 1 offset 7 | 0x3a39ce (3 bytes) -> {'temperature': 37.26, 'humidity': 57, 'wind_direction': 'E'}
```

¿QUÉ COMPLEJIDADES INTRODUCE EL TENER UN PAYLOAD RESTRINGIDO (PEQUEÑO)?

Ya no podemos mandar cualquier dato tal cual, sino que hay que estructurar el mensaje según cuánto cabe en un solo envío y decidir cuántos bits le toca a cada campo.

Esto obliga a cuidar la integridad del mensaje, producer y consumer deben interpretar los bits exactamente igual, porque cualquier diferencia corrompe los datos sin dar error. Cabe mencionar que esto permite que el uso de la red sea más ligero.



La misma lectura ocupa unas 20 veces menos.

¿CÓMO PODEMOS HACER QUE EL VALOR DE TEMPERATURA QUEPA EN 14 BITS?

Con 14 bits se pueden representar 2^14 = 16384 valores.

Multiplicamos la temperatura por 100 para enviarla como entero, de tal forma que el máximo de 110 grados centígrados se convierte en 11000, que justamente es un número que cabe en el espacio disponible. Después el consumer solo tendría que dividir entre 100 y así recuperar el valor.

14 bits para la temperatura



2^14 = 16384 valores posibles

110.00 × 100 = 11000

11000 < 16384, sí cabe

¿QUÉ SUCEDERÍA SI AHORA LA HUMEDAD TAMBIÉN ES TIPO FLOAT CON UN DECIMAL? ¿QUÉ DECISIONES TENDRÍAMOS QUE TOMAR?

Para enviar la humedad con un decimal la multiplicaríamos por 10, por lo que pasaría de 0–100 a 0–1000 y necesitaría 10 bits (2^10 = 1024) en lugar de 7. Sumando los 3 bits del viento, a la temperatura solo le quedarían 11 bits (2^11 = 2048), y el valor 11000 ya no cabe.

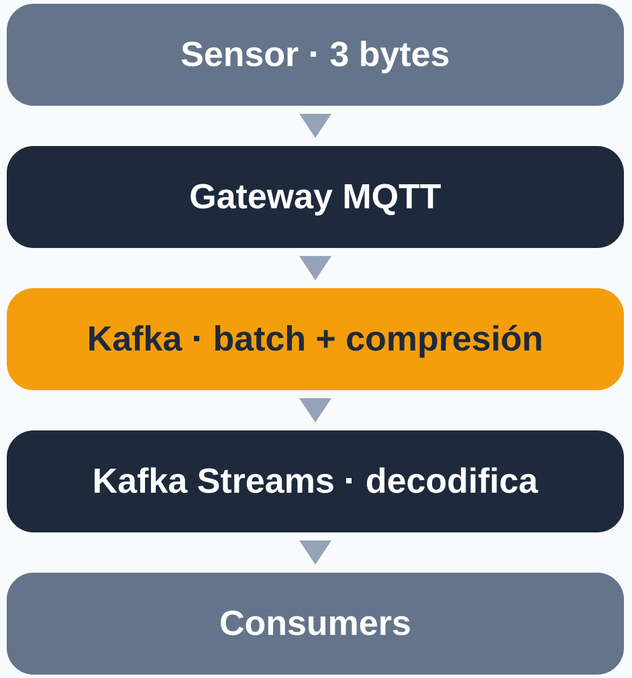
La decisión sería enviar la temperatura con un solo decimal, ya que multiplicada por 10 su máximo de 110 grados centígrados pasa a 1100, que sí cabe en 11 bits. A cambio, perdemos la segunda cifra decimal de la temperatura.



¿QUÉ PARÁMETROS O HERRAMIENTAS DE KAFKA PODRÍAN AYUDARNOS SI LAS RESTRICCIONES FUERAN AÚN MÁS FUERTES?

Si las restricciones fueran aún más fuertes, convendría que el sensor no hablara directamente con Kafka, sino con un intermediario ligero, como un endpoint MQTT, que se encargue de publicar en el broker. De esta forma, el dispositivo solo envía su formato binario compacto, como nuestros 3 bytes, y del lado del servidor se usa Kafka Streams para decodificarlo y Schema Registry con Avro para mantener un formato estable para los consumers.

Además, el producer puede agrupar varias lecturas con batch.size y linger.ms y comprimir esos lotes con compression.type.



Referencias
Yates, J. (2020). Stream Processing with IoT Data: Challenges, Best Practices, and Techniques. Confluent. https://www.confluent.io/blog/stream-processing-iot-data-best-practices-and-techniques/
Conduktor. Kafka message compression. https://www.conduktor.io/kafka/kafka-message-compression